NEWS EXPLORER

A Searchable News Repository Using Pattern Matching and Dynamic Programming

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms - 3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

Student Name: V. Navadeep	**ID: 2520030019**
Student Name: D. Namit Rao	**ID: 2520030249**
Student Name: E. Yeshwanth	**ID: 2520030539**
Team Number: 5	Section: 2

Under the Guidance of

Dr. S. Vinay Kumar

Associate Professor

Department of Computer Science and Engineering



KONERU LAKSHMAIAH EDUCATION FOUNDATION

(Deemed to be University)

Academic Year 2026–27

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

DECLARATION

We hereby declare that the project report entitled “NEWS EXPLORER” is a record of work carried out by the undersigned in partial fulfilment of the requirements for the course Data Structures and Algorithms - 3 (25CS2103E) in the Department of Computer Science Engineering during the academic year 2026–27. The report describes the design and implementation represented by the submitted Java source code and news dataset.

We further declare that the report has been prepared for academic evaluation and that any external concepts, algorithms, or documentation used are acknowledged in the References section.

Student Name: V. Navadeep ID: 2520030019

Student Name: D, Namith Rao ID: 2520030249

Student Name: E. Yeshwanth ID: 2520030539

KONERU LAKSHMAIAH EDUCATION FOUNDATION

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “NEWS EXPLORER” is a bona fide report of the project work carried out by the student team listed on the cover page, in partial fulfilment of the requirements for the course Data Structures and Algorithms - 3 (25CS2103E), Department of Computer Science Engineering, during the academic year 2026–27.

Guide	**Head of the Department**
Signature: _____ Dr. S. Vinay Kumar	Signature: _____ Dr. P. Venkateshwara Rao

External Examiner Signature: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his guidance and support throughout the development of this project. His guidance helped us connect the theoretical concepts of pattern matching and dynamic programming with a practical search application.

We also thank Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic environment and resources needed to undertake this work.

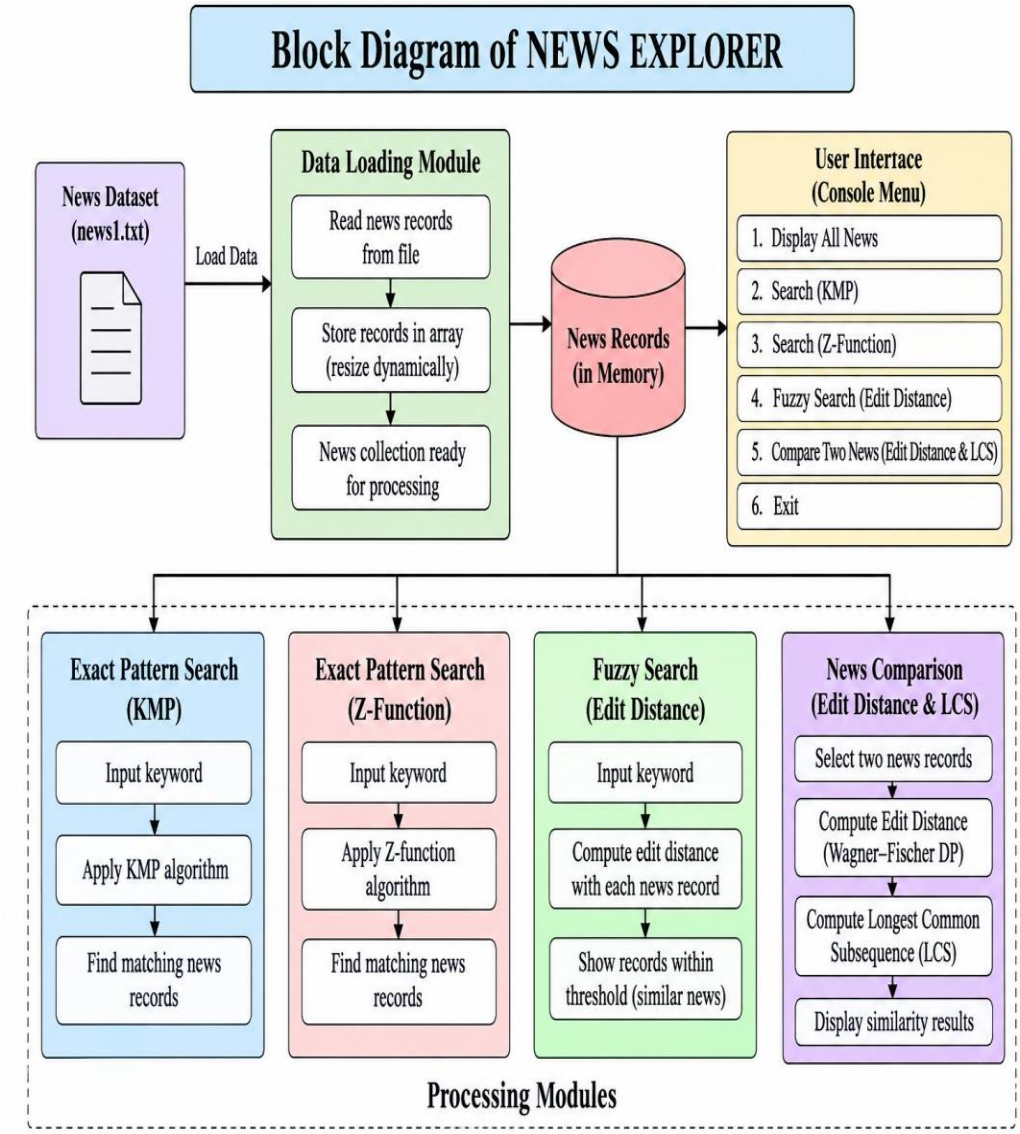
Finally, we thank our faculty members, classmates, and everyone who provided feedback during the preparation of the NEWS EXPLORER project and its documentation.

Student Name: _____ ID: _____

Student Name: _____ ID: _____

Student Name: _____ ID: _____

BLOCK DIAGRAM



ABSTRACT

NEWS EXPLORER is a console-based Java application intended to make a collection of news records easier to search and compare. The supplied dataset contains 200 numbered records with a category label and a short news headline. The application loads records from a text file, displays the collection, searches for exact character patterns using the Knuth–Morris–Pratt (KMP) algorithm or the Z-function, performs approximate word matching using edit distance, and compares two selected records using edit distance and longest common subsequence (LCS).

The project demonstrates how algorithmic techniques from Data Structures and Algorithms - 3 can support text retrieval. KMP and Z-function search avoid repeatedly comparing already matched characters, while the Wagner–Fischer dynamic programming method computes edit distance and LCS provides a measure of shared character subsequences. The current Java source is a functional prototype rather than the complete feature set proposed in the project description: it does not yet implement category filtering, hashing, tries, Rabin–Karp, or automatic related-news recommendations. The report identifies these gaps and suggests concrete improvements, including correcting the fixed-size storage capacity before loading the full 200-record dataset.

Keywords: news repository, pattern matching, KMP, Z-function, fuzzy search, edit distance, dynamic programming, LCS.

TABLE OF CONTENTS

Section	Page
Acknowledgement	i
Abstract	ii
List of Figures	iii
1. Introduction	1
1.1 Overview	1
1.2 Problem Statement	1
1.3 Scope of the Project	2
1.4 Objectives	2
2. System Requirements	3
2.1 Related Concepts and Existing Approaches	3
2.2 Functional Requirements	3
2.3 Non-Functional Requirements	4
2.4 Hardware and Software Requirements	4
3. System Design	5
3.1 High-Level Architecture	5
3.2 Data and Algorithm Design	6
3.3 Complexity Analysis	7
4. Implementation	8
4.1 Module-Wise Implementation	8
4.2 Pseudocode	9
5. Results and Testing	11
6. Challenges and Limitations	13
7. Conclusion and Future Work	14
References	16

Note: Page numbers are indicative and may shift slightly when the document is opened or printed.

LIST OF FIGURES

Figure No.	Caption
Fig. 3.1	High-level architecture of NEWS EXPLORER
Fig. 3.2	Search and fuzzy-search workflow
Fig. 5.1	Functional test cases for the implemented prototype

1. INTRODUCTION

1.1 Overview

News repositories contain large volumes of short articles and headlines across multiple topics. Finding a relevant item by manually reading every record becomes inefficient as the collection grows. NEWS EXPLORER is designed as a small searchable repository that demonstrates text-searching and similarity algorithms on a local dataset. The current implementation is a Java command-line program that reads a text file and presents a menu for browsing, searching, and comparing records.

The supplied dataset contains 200 numbered lines. Each line includes an identifier, a category label such as Technology News or Sports News, and a short headline. The dataset is suitable for demonstrating text algorithms, though its category labels and headline topics are not always semantically aligned. The program currently treats each complete line as a string rather than parsing the category and headline into separate fields.

1.2 Problem Statement

Users need a way to locate news records by a keyword without manually scanning the entire repository. A basic substring search can be inefficient if implemented with repeated naive comparisons, and exact matching can fail when the query contains a spelling error. In addition, comparing two news records can help demonstrate whether their text shares similar character sequences or requires many edits to transform one string into another.

The project addresses these needs through KMP and Z-function pattern search, edit-distance-based fuzzy search, and an article-comparison module using edit distance and LCS. A further engineering issue is that the submitted Java program allocates an array for only 50 records, while the supplied data file contains 200 records. The storage limit must be corrected for the complete dataset to load successfully.

1.3 Scope of the Project

- Load a local news text file and display the records stored in memory.

- Search the text of each record using KMP or Z-function exact pattern matching.
- Perform approximate word matching using Levenshtein edit distance with a fixed threshold of 2.
- Compare two records by reporting their edit distance and LCS length.
- Demonstrate the use of string processing and dynamic programming on a practical text-retrieval task.

The current source code does not implement category-only browsing, hash-based indexing, a trie, Rabin–Karp, or automated related-news ranking. These remain proposed extensions, not existing capabilities.

1.4 Objectives

- Build a simple and understandable news-search application in Java.
- Apply KMP and Z-function algorithms to exact substring search.
- Use dynamic programming to calculate edit distance and LCS.
- Support approximate matching for words with small spelling differences.
- Evaluate the prototype using representative functional test cases and document current limitations.

2. SYSTEM REQUIREMENTS

2.1 Related Concepts and Existing Approaches

Text search can be implemented using a naive character-by-character scan, but more structured pattern-matching algorithms can reduce redundant comparisons. KMP preprocesses a pattern into a longest-prefix-suffix (LPS) table. The Z-function computes the length of the longest substring beginning at each position that matches the prefix of a combined string. Dynamic programming is commonly used for edit distance and LCS because each larger subproblem can be built from smaller subproblems [1]–[4].

In the submitted implementation, these concepts are used directly in the application. The scope is deliberately local and educational: it uses a text file and a command-line menu rather than a database or web service. More advanced search systems often use indexes, tokenization, relevance ranking, and semantic similarity. Such features are possible future improvements but should not be considered implemented in this version.

2.2 Functional Requirements

ID	Requirement	Status in submitted code
FR-1	Read news records from a local text file.	Implemented; reads news.txt.
FR-2	Display all loaded news records.	Implemented.
FR-3	Search records with KMP.	Implemented.
FR-4	Search records with Z-function.	Implemented.
FR-5	Approximate search based on word edit distance.	Implemented; threshold <= 2.
FR-6	Compare two records using edit distance and LCS.	Implemented.
FR-7	Browse/filter records by category.	Not implemented.
FR-8	Recommend related news automatically.	Not implemented; comparison is user-selected.
FR-9	Use hashing, tries, or Rabin–Karp indexing.	Not implemented.

2.3 Non-Functional Requirements

- Usability: menu-driven interaction with clear numbered options.
- Correctness: algorithms should return expected matches for valid input and valid record numbers.
- Maintainability: separate methods are used for loading, display, search, fuzzy matching, and comparison.
- Performance: KMP and Z-function provide linear-time pattern matching after/while processing the text; edit-distance computation uses a two-dimensional table.
- Reliability: the program should validate file availability, input values, empty keywords, and record capacity. Some of these validations are not yet present in the submitted source.
- Scalability: the current fixed array limits capacity to 50 records; dynamic storage is required for larger collections.

2.4 Hardware and Software Requirements

Type	Specification
Hardware	Any standard desktop or laptop capable of running a Java runtime; recommended 4 GB RAM or more.
Operating system	Windows, Linux, or macOS.
Language	Java.
Runtime / compiler	JDK compatible with the source code; Java standard I/O libraries are used.
Input data	Plain-text news file containing one record per line.
Execution interface	Command-line terminal.

The source opens a file named news.txt in the program working directory. The uploaded data file is named news1(1).txt, so it must be renamed/copied to news.txt or the Java path must be changed. Also, the news array must be enlarged

or replaced with a dynamic collection before running against all 200 supplied records.

3. SYSTEM DESIGN

3.1 High-Level Architecture

The application follows a simple layered flow: a text file provides input; the loading module stores records in memory; a menu accepts the user's operation; the selected algorithm processes the records; and matching lines or comparison scores are printed to the console.

Layer	Responsibility
Input layer	Reads one news record per line from news.txt.
Storage layer	Stores records in the static String array news and tracks the count.
Control layer	Displays menu choices and dispatches to the selected method.
Algorithm layer	Runs KMP, Z-function, edit distance, or LCS.
Output layer	Prints records, search matches, or comparison values.

Fig. 3.1 High-level architecture of NEWS EXPLORER

Figure 3.1. The system is a local console application; it has no separate frontend, backend server, database, or external API.

3.2 Data and Algorithm Design

3.2.1 Dataset Representation

Each source line is stored as a single String. A record resembles "1 | Technology News | Technology researchers unveil a new generation of energy-efficient computing systems". The current code does not split the identifier, category, and headline into independent fields. Consequently, exact and fuzzy search can match words anywhere in the line, including the category label or record number.

3.2.2 KMP Pattern Matching

KMP preprocesses the query pattern to build an LPS array. When a mismatch occurs, the algorithm uses the LPS value to shift the pattern without restarting the comparison from the beginning. Its time complexity is O(n + m), where n is the

text length and m is the pattern length [1]. The implementation returns true as soon as the complete pattern is found.

3.2.3 Z-Function Search

The Z-function method constructs pattern + "$" + text and calculates Z-values. A position whose Z-value equals the pattern length indicates a match. The combined string must use a separator that does not occur in the input pattern or text; the current implementation assumes "$" is safe. Its time complexity is O(n + m) under the usual Z-algorithm assumptions [2].

3.2.4 Fuzzy Search with Edit Distance

The Wagner–Fischer dynamic programming algorithm calculates the minimum number of insertions, deletions, and substitutions required to transform one string into another. For each record, the implementation splits the lowercased line into words, calculates the distance from the query to each word, and prints the record when the minimum distance is at most 2. For strings of lengths n and m, the standard algorithm takes O(nm) time and O(nm) space [3].

3.2.5 Article Comparison with LCS

The comparison module accepts two record numbers and computes their character-level edit distance and LCS length. Edit distance gives a transformation cost; LCS gives the length of a longest sequence of characters appearing in both strings in the same order. Both implementations use two-dimensional dynamic programming tables and take O(nm) time and space for input lengths n and m [3], [4].

3.3 Complexity Analysis

Operation	Time complexity	Auxiliary space	Notes
Load/display records	O(N)	O(N) storage	N is number of records; each record is stored as a String.
KMP search per record	O(L + M)	O(M)	L = record length; M = query length.
Z-function search per record	O(L + M)	O(L + M)	Builds combined string and Z array.

Fuzzy search per record	O(W × Q × A)	O(Q × A)	W words; Q query length; A typical word length; each word uses edit-distance DP.
Edit distance for two records	O(nm)	O(nm)	Two-dimensional DP table.
LCS for two records	O(nm)	O(nm)	Two-dimensional DP table.

For a repository of N records, exact search scans each record, so total search time is the sum of the per-record costs. Indexing structures could reduce repeated full scans, but they are outside the current implementation.

4. IMPLEMENTATION

4.1 Module-Wise Implementation

Module / method	Purpose
loadNews()	Reads non-empty lines from news.txt into the news array.
showNews()	Prints every record currently loaded.
lps(pattern)	Builds the LPS table used by KMP.
kmp(text, pattern)	Returns true if the pattern occurs in the text.
zSearch(text, pattern)	Uses the Z-function on pattern + separator + text.
editDistance(a, b)	Computes Levenshtein edit distance using dynamic programming.
lcs(a, b)	Computes the length of the longest common subsequence.
search(keyword, type)	Runs KMP when type is 1; otherwise uses Z-function search.
fuzzySearch(keyword)	Checks each word and prints records whose best word distance is <= 2.
compareNews()	Reads two record numbers and prints edit distance and LCS length.
main()	Loads data, presents the menu, and dispatches operations.

4.2 Pseudocode

Algorithm 1: Load news records

```
OPEN input file news.txt
FOR each line in file:
    IF line is not blank:
        store line in news[count]
        count = count + 1
CLOSE file
```

Algorithm 2: Exact search

```
CONVERT keyword to lowercase
FOR each stored news record:
    CONVERT record to lowercase
    IF selected algorithm (KMP or Z) finds keyword:
        PRINT record
```

Algorithm 3: Fuzzy search

```
CONVERT query to lowercase
FOR each news record:
    split record into words
    bestDistance = infinity
    FOR each word:
        remove basic comma/period punctuation
        d = editDistance(query, word)
        bestDistance = MIN(bestDistance, d)
    IF bestDistance <= 2:
        PRINT record
```

Algorithm 4: Compare two records

```
READ article number a and article number b
x = news[a - 1]; y = news[b - 1]
PRINT editDistance(x, y)
PRINT lcs(x, y)
```

4.3 Execution Procedure

1. Place the dataset in the program working directory and name it news.txt, or update the file path in loadNews().
2. Ensure the storage capacity is sufficient for the number of input lines. The submitted dataset has 200 lines, but the current array has capacity 50.
3. Compile the source with a compatible JDK, for example: javac NewsExplorer.java.
4. Run the application, for example: java NewsExplorer.
5. Choose menu options 1–6 to display records, perform KMP search, perform Z-function search, run fuzzy search, compare two records, or exit.

4.4 Important Implementation Notes

The submitted source uses a fixed array of 50 entries and does not check the capacity before writing. With the supplied 200-line dataset, loading will attempt to write beyond the array and can terminate with an ArrayIndexOutOfBoundsException. Replace the array with an ArrayList<String> or allocate sufficient capacity and add a bounds check. The file-name mismatch must also be resolved. The program does not currently normalize all punctuation, validate article numbers, handle an empty search pattern safely, or guarantee that "$" is absent from the searched text.

5. RESULTS AND TESTING

The results in this section describe expected functional behavior from inspection of the submitted source and dataset. They are test cases to execute after correcting the file-name and storage-capacity issues; they are not claimed as measured runtime benchmarks. Exact output depends on the chosen query and the records present in the text file.

5.1 Test Cases

Test ID	Input / action	Expected behavior	Status / note
TC-01	Select Show News (option 1).	Print all loaded records in order.	Requires successful data loading.
TC-02	KMP query: “cloud”.	Print records containing “cloud” regardless of case.	Should match relevant lines such as cloud-computing headlines.
TC-03	Z-function query: “cricket”.	Print records containing “cricket” regardless of case.	Search scans complete lines, including category text.
TC-04	Fuzzy query: “computng”.	Print records containing a word within edit distance 2 of the query, if any.	Demonstrates typo tolerance; output depends on word lengths.
TC-05	Compare two valid article numbers.	Print edit distance and LCS length.	Values depend on selected records.
TC-06	Run against the supplied 200-line file with unchanged code.	Loading attempts to exceed the 50-element array.	Known defect; increase capacity first.
TC-07	Run without a file named news.txt.	File opening fails with an I/O exception.	Rename the dataset or update the path.
TC-08	Enter an invalid article number or non-integer menu choice.	Input parsing/indexing may fail.	Input validation should be added.

5.2 Functional Interpretation

The source code includes two exact search paths and one approximate word-search path. The comparison module reports two complementary string metrics.

These features illustrate different DSA ideas: pattern preprocessing in KMP, prefix matching in the Z-function, and optimal substructure in dynamic programming. The prototype is suitable for demonstrating these algorithms once the data-loading issue is fixed.

5.3 Figures

Fig. 5.1 Functional test plan for the implemented prototype

Test group	Representative checks
Data handling	File found; records loaded; all lines fit storage.
Exact search	KMP and Z-function find existing keywords and return no match for absent keywords.
Approximate search	A small typo can match a nearby word; a distant spelling should not pass the threshold.
Comparison	Edit distance is zero for identical strings; LCS length equals string length for identical strings.
Input robustness	Invalid choices, empty queries, and out-of-range record numbers are handled safely.

Figure 5.1. The test plan groups verification into data handling, exact search, fuzzy matching, comparison, and input robustness. No performance graph is included because no timing measurements were supplied.

6. CHALLENGES AND LIMITATIONS

6.1 Issues Identified

- Storage capacity mismatch: the Java array has 50 slots, while the supplied dataset contains 200 records.
- Input filename mismatch: the code expects news.txt, while the uploaded data file is named news1(1).txt.
- Whole-line matching: record ID, category, and headline are stored together, so queries can match metadata as well as headline text.
- Limited fuzzy matching: fuzzy search compares the query against individual words, not complete phrases or article meaning.
- Limited punctuation handling: only periods and commas are removed from words in fuzzy search.
- Input validation: article numbers and numeric menu choices are parsed without comprehensive validation.
- Feature gap: category filtering, hash indexing, tries, Rabin–Karp, and automated related-news recommendation are not present in the submitted source.
- Dataset quality: some category labels do not correspond closely to the headline topic, which limits the reliability of category-based interpretation.

6.2 Solutions and Improvements

- Replace String[50] with ArrayList<String>, or dynamically resize storage as records are read.
- Use the correct file path or accept a file name as a command-line argument.
- Create a NewsArticle class with separate id, category, and headline fields.
- Validate menu input and article numbers, and handle empty queries and file errors gracefully.
- Normalize case and punctuation consistently before matching.
- Add unit tests for known KMP, Z-function, edit-distance, and LCS examples.
- Implement category filtering with parsed category fields, then consider a trie or hash-based index for faster repeated queries.

- Implement related-news suggestions by computing a similarity score and ranking the most similar records; clearly define and evaluate the score.

6.3 Limitations of the System

NEWS EXPLORER currently works as a local command-line prototype and does not retrieve live news from the internet. It does not store metadata in a database, provide a graphical interface, or rank results by relevance. The edit-distance method can be computationally expensive for long strings and can return false positives when common short words are similar. LCS and edit distance are character-level metrics here and should not be interpreted as semantic similarity. The reported features and conclusions are therefore limited to the code and sample dataset provided.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

NEWS EXPLORER demonstrates a practical application of pattern matching and dynamic programming on a local collection of news headlines. The Java program provides record display, KMP search, Z-function search, approximate word matching using edit distance, and pairwise comparison using edit distance and LCS. These methods help illustrate algorithm design, preprocessing, recurrence relations, and time/space complexity.

The supplied dataset contains 200 records, which exposes a storage-capacity defect in the submitted program because its array holds only 50 entries. The file path also needs to be aligned with the expected name news.txt. Once these issues are addressed, the core search and comparison modules can be tested on the full sample collection. The project description's additional goals—category browsing, hashing, tries, Rabin–Karp, and related-news recommendation—should be treated as future work until implemented and validated.

7.2 Future Work

- Use dynamic storage and parse each record into structured fields.
- Add category-based browsing and filters.
- Implement Rabin–Karp and compare its behavior with KMP and Z-function search.
- Use a trie for prefix-based search and hashing for fast lookup or deduplication.
- Develop a related-news recommendation module using token overlap, cosine similarity, or a carefully defined edit-distance-based score.
- Rank results by relevance and show a short explanation for each match.
- Add a graphical or web interface, persistent storage, and support for larger datasets.
- Measure runtime on datasets of different sizes and present reproducible performance graphs.

7.3 Skills Learned During Development

- Implementing string-matching algorithms in Java.

- Using dynamic programming for edit distance and LCS.
- Reading structured text records from files.
- Designing modular console applications and menu-driven control flow.
- Analyzing complexity, testing edge cases, and documenting software limitations.

REFERENCES

[1] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[2] D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge University Press, 1997.

[3] R. A. Wagner and M. J. Fischer, “The String-to-String Correction Problem,” Journal of the ACM, vol. 21, no. 1, pp. 168–173, 1974.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. MIT Press, 2009.

[5] Oracle, “Java Platform, Standard Edition Documentation.” https://docs.oracle.com/en/java/

[6] Project source code: NewsExplorer.java, supplied for the NEWS EXPLORER project.

[7] Project dataset: news1(1).txt, supplied with the NEWS EXPLORER project.

[8] DSA-3 Project Documentation Format 2026–27 and Guidelines for Project Document, supplied course documents.